



Università di Genova

High Performance Computing Report Mandelbrot Algorithm

Andrea Cattarinich 5137057

December 3, 2025

Contents

1	Architectures	3
1.1	Personal Workstation	3
1.2	University Workstation	3
1.3	Google Colab	4
2	Mandelbrot Set	5
2.1	Definition	5
2.1.1	Point that belongs to the Mandelbrot set	5
2.1.2	Point that does NOT belong to the Mandelbrot set	5
2.2	Images	5
3	Hotspot Analysis	6
3.1	Optimization Problems	6
3.1.1	Solution and Results	6
3.2	Hotspot Identification	7
3.3	Conclusions	8
4	OpenMP	9
4.1	Parallel For	9
4.2	Guided Scheduling	10
4.3	Guided + SIMD	11
4.4	Conclusions	11
5	CUDA	12
5.1	Code	12
5.2	Compilation	13
5.3	Results	13

6	Conclusions	14
6.1	CUDA vs OpenMP	14

1 Architectures

For this project I have used three workstation, my personal one, the workstation provided by the University and Google Colab.

1.1 Personal Workstation

HP Envy 17-ae103nl

CPU

Model	Intel Core i7-6500U @ 2.50 GHz
Architecture	x86_64
Physical cores	2
Logical threads	4 (Hyper-Threading)

GPU

Model	NVIDIA GeForce GTX 950M
CUDA Compute Capability	5.0
Driver version	572.16
CUDA Version	12.8

Software and Setup

Visual Studio	Visual Studio Community 2022 (C++ Desktop workload)
CUDA Toolkit	Installed manually
WSL2	Enabled
NVIDIA Drivers	Version 572.16 (for WSL2)

1.2 University Workstation

Room 210

CPU

Model	Intel Core i9-12900K (12th Gen)
Architecture	x86_64
CPU(s)	24
Threads per core	2
Cores per socket	16
Socket(s)	1
Max frequency	4.02 GHz
Profiling tools	Intel VTune Profiler, Intel Advisor

To use the Intel oneAPI tools, the environment was loaded with:

```
source /opt/intel/oneapi/setvars.sh
```

1.3 Google Colab

CPU

Model	Intel(R) Xeon(R) CPU @ 2.20GHz
Architecture	x86_64
Physical cores	1
Logical threads	2 (Hyper-Threading)
Socket(s)	1

GPU

Model	NVIDIA Tesla T4
CUDA Compute Capability	7.5
Driver version	550.54.15
CUDA Version	12.8

2 Mandelbrot Set

2.1 Definition

The Mandelbrot fractal is defined as a set of points in the complex plane. Each point corresponds to a complex number c .

For every c , we consider the iterative sequence:

$$z_0 = 0, \quad z_{n+1} = z_n^2 + c$$

We then observe the behavior of z_n as $n \rightarrow \infty$.

2.1.1 Point that belongs to the Mandelbrot set

Take $c = -1$. Then the sequence is

$$z_0 = 0, \quad z_1 = 0^2 + (-1) = -1, \quad z_2 = (-1)^2 + (-1) = 0, \quad z_3 = 0^2 + (-1) = -1, \dots$$

The results remain bounded, so $c = -1$ belongs to the Mandelbrot set.

2.1.2 Point that does NOT belong to the Mandelbrot set

Take $c = 1$. Then the sequence is

$$z_0 = 0, \quad z_1 = 0^2 + 1 = 1, \quad z_2 = 1^2 + 1 = 2, \quad z_3 = 2^2 + 1 = 5, \dots$$

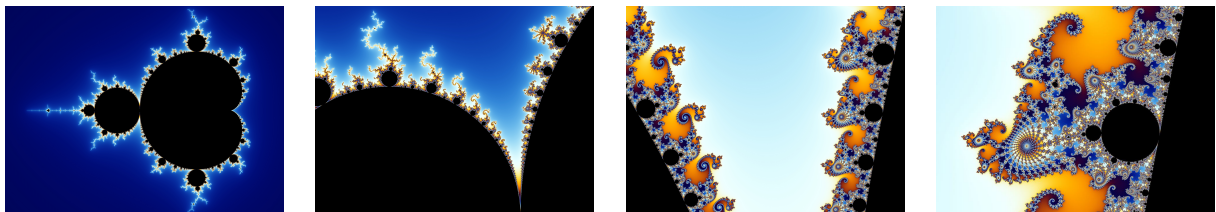
The values grow up to infinite, so $c = 1$ does not belong to the Mandelbrot set.

In our specific case, the escape condition is defined as:

$$|z_i| > 2$$

2.2 Images

This process gives rise to the famously intricate structure of the Mandelbrot fractal. Here a simple explanation [1] or here a more complex one [2] and here an online demo [3].



The color represents how quickly the values of z_n grow:

Black - the sequence remains bounded;

Other colors - the sequence escapes to infinity, with the color indicating how fast it diverges.

3 Hotspot Analysis

3.1 Optimization Problems

Questa istruzione non e' vettorizzabile con MSVC, e per tanto, l'esecuzione risulta essere poco performante.

Mentre con Linux G++, l'operazione sembra ottimizzare `std::complex`.

```
1 z = pow(z, 2) + c;
```

L'operazione `std::complex` utilizza l'overloading, in particolare `operator+` e `operator*`, creando un overhead e un conseguente carico sulla CPU. Inoltre, l'operazione `abs(z)` calcola la radice quadrata ogni volta.

Per rimuovere questa dipendenza dovuta dai numeri complessi, ho realizzato un miglioramento nel codice che mi consente di usare solamente numeri reali. Infatti, sfruttando la seguente relazione

$$z^2 = (a, b)^2 = (a^2 - b^2, 2ab)$$

Ho implementato:

$$z^2 = (\text{Re}(z), \text{Im}(z))^2 = (\text{Re}(z)^2 - \text{Im}(z)^2, 2 \text{Re}(z) \text{Im}(z))$$

Come segue:

```
1 double zRe = 0, zIm = 0;
2 for (int i = 1; i <= ITERATIONS; i++)
3 {
4     double oldRe = zRe;
5     double zRe2 = zRe*zRe; // Re(z)^2
6     double zIm2 = zIm*zIm; // Im(z)^2
7
8     zRe = (zRe2 - zIm2) + cRe;
9     zIm = 2*oldRe*zIm + cIm;
10
11     if(zRe2 + zIm2 >= 4)
12     {
13         image[pos] = i;
14         break;
15     }
16 }
```

3.1.1 Solution and Results

I risultati, come ci aspettavamo sono più performanti.

MSVC

```
cl /O2 /arch:AVX2 /fp:fast /EHsc src\*.cpp /Fe:build\*.exe
```

WSL

```
g++ -O2 -o build/* src/*.cpp
```

	mandelbrot	opt1	opt2
WSL	96s	30s (x3.2)	36s (x2.7)
Windows	-	40s	36 s

Utilizzeremo questo codice come baseline per lo sviluppo della restante parte del progetto.

3.2 Hotspot Identification

The measurements were performed using Intel VTune Profiler and the analysis was performed using Intel Advisor.

Function Stack	Total	Percentage (%)
Total	69.97	100.00
_start	69.97	100.00
__libc_start_main_impl	69.97	100.00
- main	69.97	100.00
— std::abs<double>	41.2121	58.90
— std::operator+<double>	13.6361	19.49
— std::pow<double>	10.712	15.31
— std::complex<double>::complex	0.0199997	0.03

The two main hotspots are the following instructions which takes the most fraction of the program execution.

Line 48

```
1 z = pow(z, 2) + c;
```

Line 51

```
1 if (abs(z) >= 2)
```

Source Line ▲	Source	🔥 CPU Time: Total	CPU Time: Self
31			
32	const auto start = chrono::steady_clock::now();		
33	for (int pos = 0; pos < HEIGHT * WIDTH; pos++)	0.0%	0.008s
34	{		
35	image[pos] = 0;	0.0%	0.012s
36			
37	const int row = pos / WIDTH;	0.0%	0.008s
38	const int col = pos % WIDTH;	0.0%	0.012s
39	const complex<double> c(col * STEP + MIN_X, row * STEP + MIN_Y);	0.0%	0.012s
40			
41	// z = z^2 + c		
42	complex<double> z(0, 0);		
43	for (int i = 1; i <= ITERATIONS; i++)		
44	{		
45	z = pow(z, 2) + c;	39.9%	3.570s
46			
47	// If it is convergent		
48	if (abs(z) >= 2)	59.9%	0.732s
49	{		
50	image[pos] = i;		
51	break;		
52	}		
53	}	0.0%	0.024s
54	}	0.0%	0.012s

3.3 Conclusions

The main hotspot is the DFT() function and its inverse IDFT(), which together account for over 99,05% of the total execution time. All other functions have negligible impact. This indicates that **parallelization and vectorization efforts should focus on the double loop inside DFT()**.

The main hotspot is the loop contained in lines 72-80 (source: src/omp_homework.c). The dominant operations contributing to the algorithm's computational intensity are the `sin()` and `cos()` trigonometric functions, which account for the majority of the execution time, as shown in the following code.

Source Line ▲	Source	🔥 CPU Time: Total ⓘ	CPU Time: Self ⓘ
31			
32	const auto start = chrono::steady_clock::now();		
33	for (int pos = 0; pos < HEIGHT * WIDTH; pos++)	0.0%	0.008s
34	{		
35	image[pos] = 0;	0.0%	0.012s
36			
37	const int row = pos / WIDTH;	0.0%	0.008s
38	const int col = pos % WIDTH;	0.0%	0.012s
39	const complex<double> c(col * STEP + MIN_X, row * STEP + MIN_Y);	0.0%	0.012s
40			
41	// z = z^2 + c		
42	complex<double> z(0, 0);		
43	for (int i = 1; i <= ITERATIONS; i++)		
44	{		
45	z = pow(z, 2) + c;	39.9%	3.570s
46			
47	// If it is convergent		
48	if (abs(z) >= 2)	59.9%	0.732s
49	{		
50	image[pos] = i;		
51	break;		
52	}		
53	}	0.0%	0.024s
54	}	0.0%	0.012s

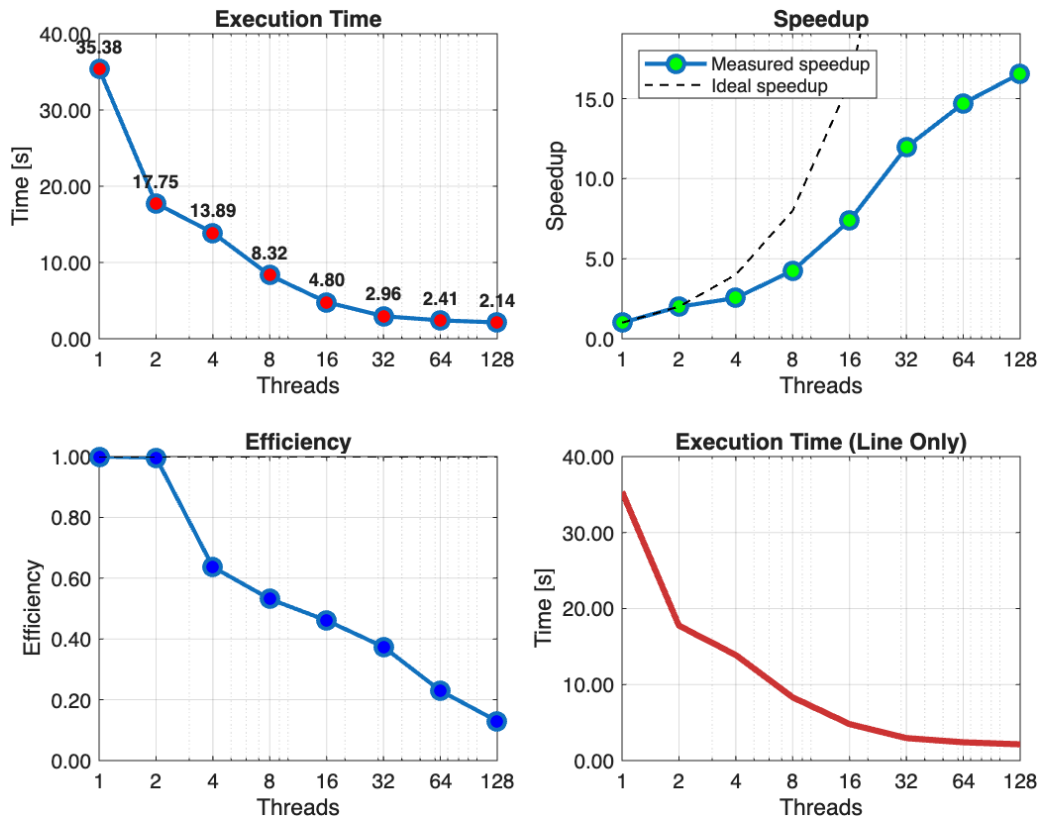
4 OpenMP

All results were obtained by compiling with the following command (see Makefile) running each version 10 times, and computing the average runtime: `icpx -O2 -march=native -fopenmp`

4.1 Parallel For

In the first approach (`src/omp1.cpp`), the outer loop over was parallelized by the directive using default static scheduling (`schedule(static)`). Evenly divides pixels among available threads.

```
1  #pragma omp parallel for
2  for (int pos = 0; pos < HEIGHT * WIDTH; pos++)
3  {
4      image[pos] = 0;
5      const int row = pos / WIDTH;
6      const int col = pos % WIDTH;
7      double cRe = col * STEP + MIN_X;
8      double cIm = row * STEP + MIN_Y;
9      double zRe = 0, zIm = 0;
10     for (int i = 1; i <= ITERATIONS; i++)
11     {
12         double oldRe = zRe;
13         zRe = (zRe*zRe - zIm*zIm) + cRe;
14         zIm = 2*oldRe*zIm + cIm;
15
16         if(zRe*zRe + zIm*zIm >= 4)
17         {
18             image[pos] = i;
19             break;
20         }
21     }
22 }
```



Execution times decrease monotonically as the number of threads increases, and the corresponding speedup grows accordingly. As expected, it remains below the theoretical Amdahl limit (considering an almost negligible sequential fraction). Efficiency is high only for the first few threads (1 up to 2), after which load imbalance becomes noticeable.

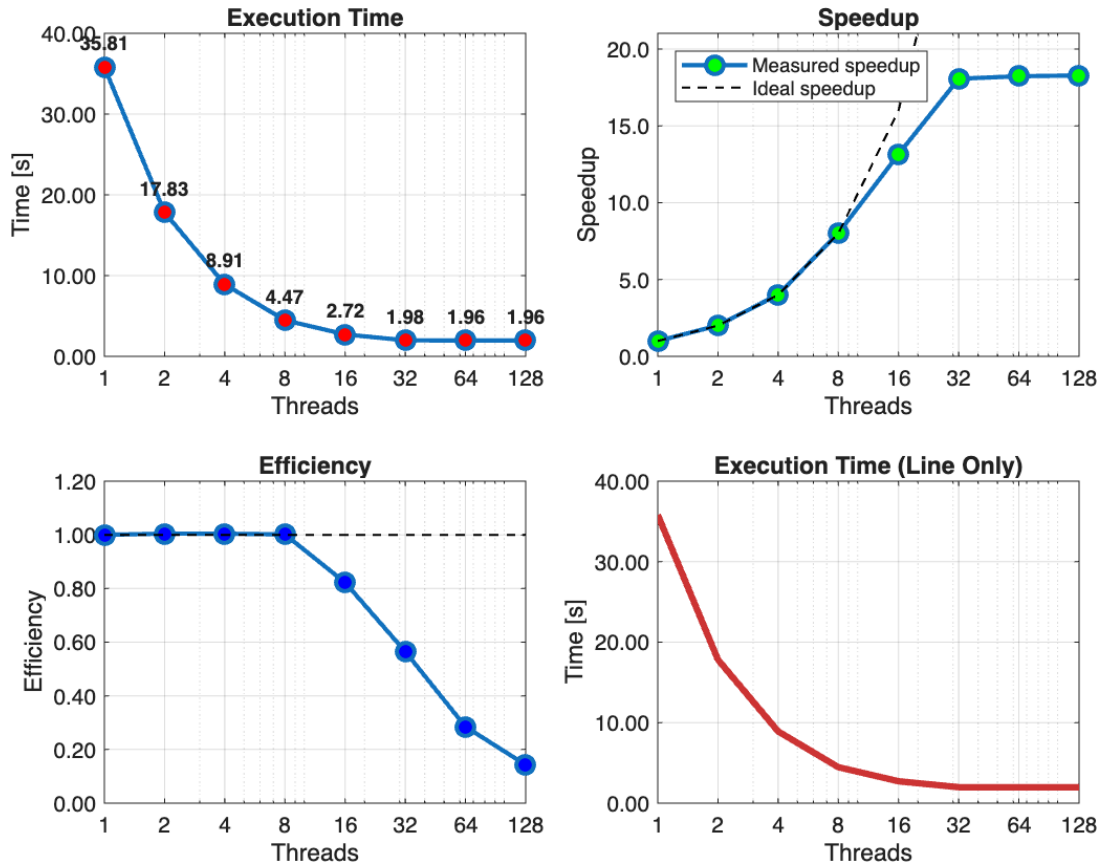
4.2 Guided Scheduling

The following scheduling policy dynamically balances workload distribution. It is essential for handling the computational irregularity of the Mandelbrot set algorithm because different pixels requires different amount of iterations.

```

1  #pragma omp parallel for schedule(guided)
2  for (int pos = 0; pos < HEIGHT * WIDTH; pos++)
3  {
4      // ... same content ...
5  }

```

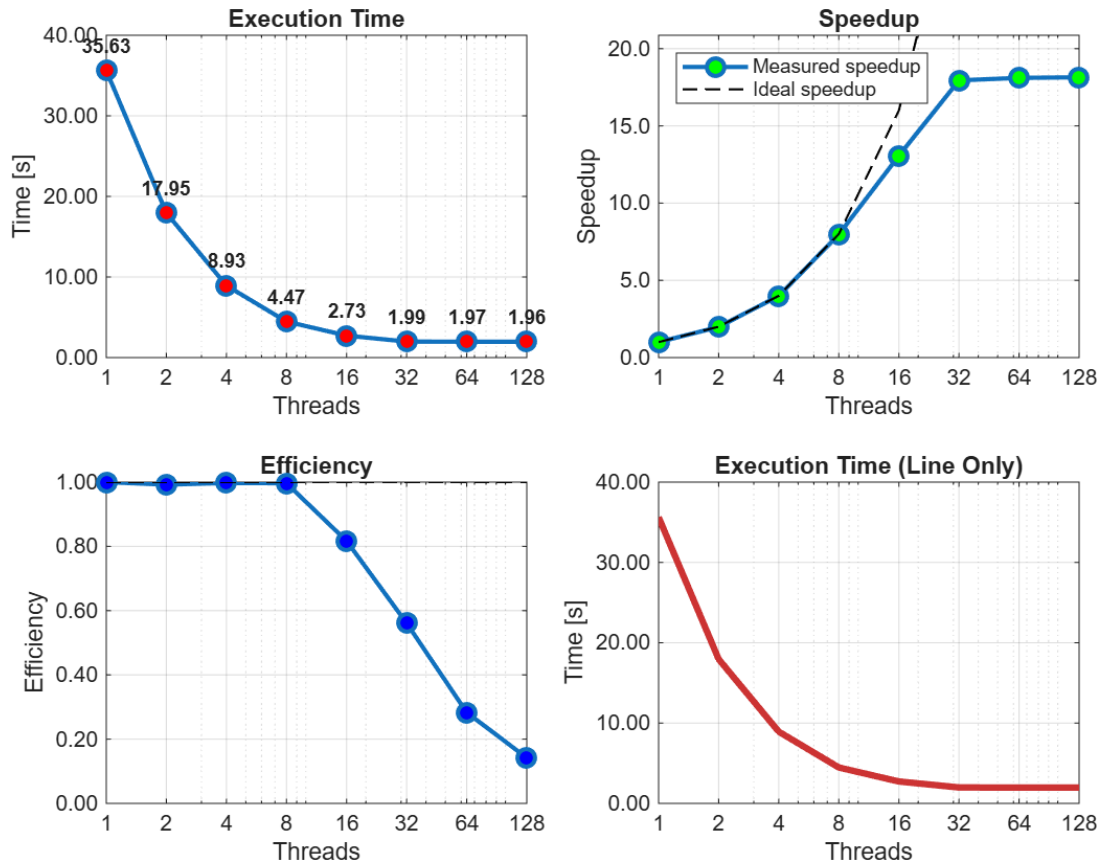


Guided scheduling provides clear improvements in both speedup and efficiency. The measured speedup remains close to the theoretical value up to 16 threads (theoretical speedup 16, observed ≈ 13), corresponding to an efficiency of about 80%. This demonstrates that guided scheduling effectively mitigates the workload irregularity.

4.3 Guided + SIMD

Maximum optimization combining multi-thread parallelism with SIMD vectorization.

```
1  #pragma omp parallel for simd schedule(guided)
2  for (int pos = 0; pos < HEIGHT * WIDTH; pos++)
3  {
4      // ... same content ...
5  }
```



The `simd` clause suggests vectorization, but the Mandelbrot loop includes divergent control flow, which may hinder full vectorization. This is reflected in the results: performance remains essentially unchanged compared to the guided-only version, confirming the difficulty of achieving meaningful SIMD gains in this context.

4.4 Conclusions

The static scheduling strategy is not ideal for the Mandelbrot computation because the workload is highly irregular: different pixels require significantly different iteration counts. Guided scheduling, instead, adapts more effectively to this variability and leads to better load balancing.

5 CUDA

I implemented a CUDA version of the Mandelbrot computation and compared it with a CPU baseline, based on the results obtained with my PC.

I defined the parameters as follows: *RESOLUTION* = 5000 and *ITERATIONS* = 100, resulting in an image of $15,000 \times 10,000$ pixels (approximately 150 million points).

5.1 Code

Memory allocation: this step prepares the memory needed for the Mandelbrot output.

```
1  const int N = WIDTH * HEIGHT;
2  int *image_host = new int[N];
3  int *image_dev;
4
5  cudaMalloc(&image_dev, N * sizeof(int));
```

Kernel configuration and execution: the code defines the CUDA execution parameters: a block of 256 threads and a 1D grid large enough to cover all *N* pixels.

```
1  dim3 block(256);
2  dim3 grid((N + block.x - 1) / block.x);
3
4  mandelbrot_kernel_1D<<<grid, block>>>(
5      image_dev, WIDTH, HEIGHT, STEP, ITERATIONS, MIN_X, MIN_Y
6  );
7
8  cudaDeviceSynchronize();
```

Kernel Mandelbrot 1D: the `mandelbrot_kernel_1D()` is a GPU kernel that parallelizes the Mandelbrot computation by assigning one thread per pixel.

```
1  __global__ void mandelbrot_kernel_1D(int *image, int width, int height,
2                                     double step, int iterations,
3                                     double min_x, double min_y)
4  {
5      int pos = blockIdx.x * blockDim.x + threadIdx.x;
6      int size = width * height;
7
8      if (pos >= size) return;
9
10     int row = pos / width;
11     int col = pos % width;
12
13     double cRe = col * step + min_x;
14     double cIm = row * step + min_y;
15
16     double zRe = 0, zIm = 0;
17
18     for (int i = 1; i <= iterations; i++)
19     {
20         double oldRe = zRe;
21         zRe = (zRe*zRe - zIm*zIm) + cRe;
22         zIm = 2 * oldRe * zIm + cIm;
23
24         if (zRe*zRe + zIm*zIm >= 4.0)
```

```

25     {
26         image[pos] = i;
27         return;
28     }
29 }
30
31 image[pos] = 0;
32 }

```

Copy-back and cleanup: After the GPU finishes, the results are copied from device to host using:

```

1     cudaMemcpy(image_host, image_dev, N * sizeof(int),
2         cudaMemcpyDeviceToHost);
3     cudaFree(image_dev);

```

5.2 Compilation

The compilation commands used for the different architectures were the following:

Personal PC:

```

CPU: g++ -O2 -o buildHP/opt1 src/opt1.cpp
GPU: nvcc -O2 -arch=sm_50 src/*.cu -o buildHP/*

```

Google Colab:

```
!nvcc -O2 -arch=sm_75 -o "$ROOT/build/*" "$ROOT/src/*.cu"
```

5.3 Results

Both 1D and 2D CUDA kernels were tested, but they produced identical results, so I considered only the 1D version.

Additionally, a version of the kernel using **float precision** (code in: `src/mandelbrot_float.cu`) was tested, providing a significant speedup on GPU. The float kernel uses *float* instead of *double* for all real-number calculations, reducing memory usage and improving throughput on the GPU.

Kernel	CPU	GPU 950M	GPU T4
1D	24668 ms	1768 ms (13x)	506 ms (48x)
Float	-	194 ms (127x)	27.8 ms (887x)

6 Conclusions

6.1 CUDA vs OpenMP

For completeness, I also compared the CUDA results with the CPU OpenMP implementation. The GPU implementations, especially the float kernel, which is the faster implementation, provide the following speedup.

References

- [1] Simple Explanation of the Mandelbrot Set.
<https://www.wikihow.com/Plot-the-Mandelbrot-Set-By-Hand>
- [2] Mandelbrot Set – Formal Explanation.
https://en.wikipedia.org/wiki/Mandelbrot_set
- [3] Online Mandelbrot Demo.
<https://math.hws.edu/eck/js/mandelbrot/MB.html>